MODEL-1

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

In [31]:
```
df=pd.read_csv('categorised_e11.csv')
```

In [32]:
```
# categorising the columns to predict into different levels

df['alarm']=0
df['alert']=0

df['alarm'] = np.where((df['c51'] <= 5) & (df['c53'] <= 5), 0,
                np.where((df['c51'] <= 10) & (df['c53'] <= 10), 0,
                np.where((df['c51'] <= 20) & (df['c53'] <= 20), 0,1)))
df['alert'] = np.where((df['c51'] <= 5) & (df['c53'] <= 5), 0,
                np.where((df['c51'] <= 10) & (df['c53'] <= 10), 0,
                np.where((df['c51'] <= 20) & (df['c53'] <= 20), 1,0)))
```

In [33]:
```
# selecting 30 best features for model
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

X = df.drop(columns=['c51','c52','c53','c54','alert','alarm'])
y = df['alert']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

k = 30
selector = SelectKBest(chi2, k=k)
X_train_selected = selector.fit_transform(X_train, y_train)

selected_features = X.columns[selector.get_support()]

print('Selected Features:', selected_features)
```

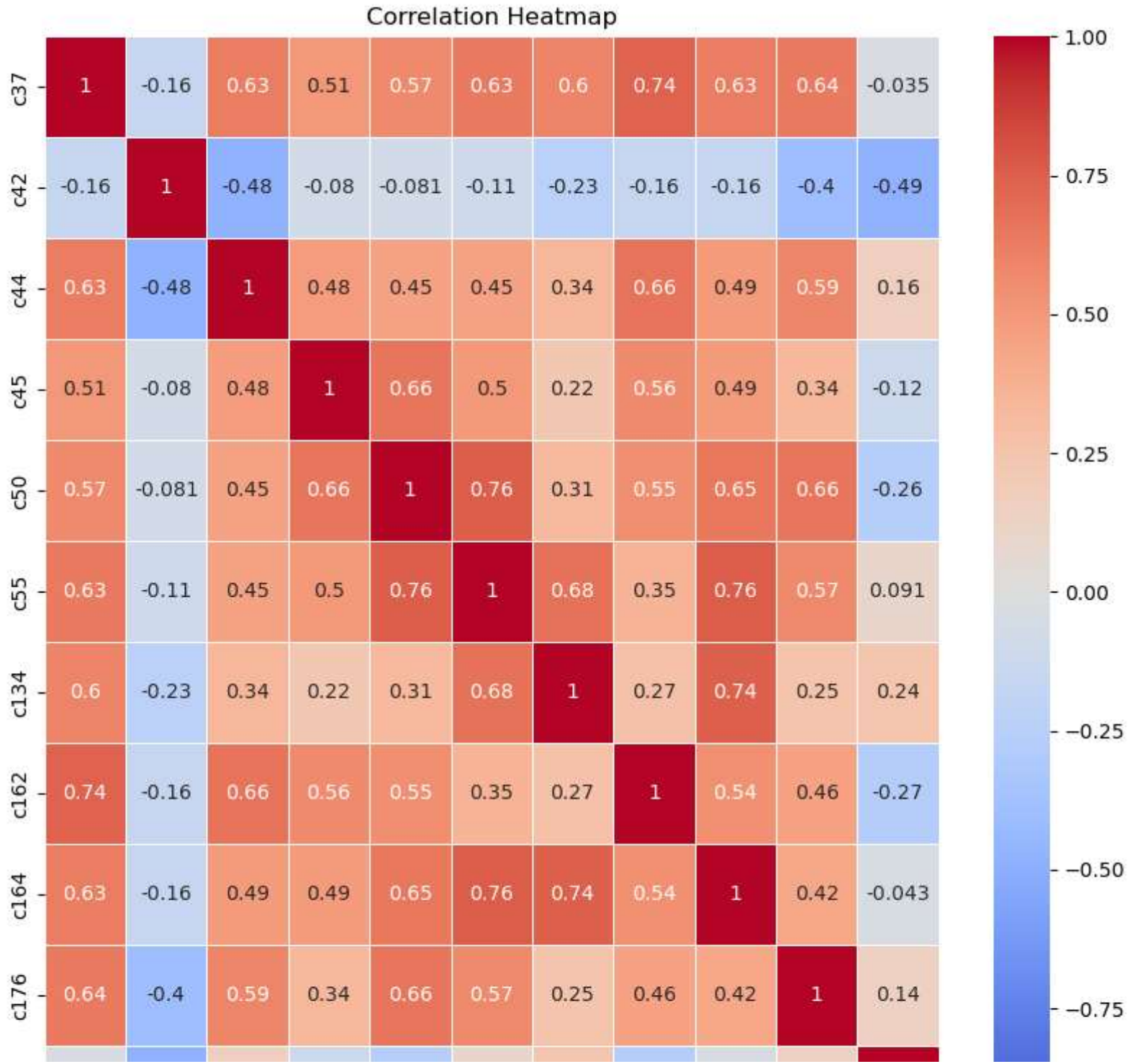
```
Selected Features: Index(['c37', 'c42', 'c44', 'c45', 'c50', 'c55', 'c87', 'c89', 'c112', 'c118',
       'c134', 'c135', 'c126', 'c130', 'c131', 'c150', 'c155', 'c159', 'c161',
       'c162', 'c164', 'c176', 'c177', 'c178', 'c185', 'c193', 'c197', 'c203',
       'c236', 'c237'],
      dtype='object')
```

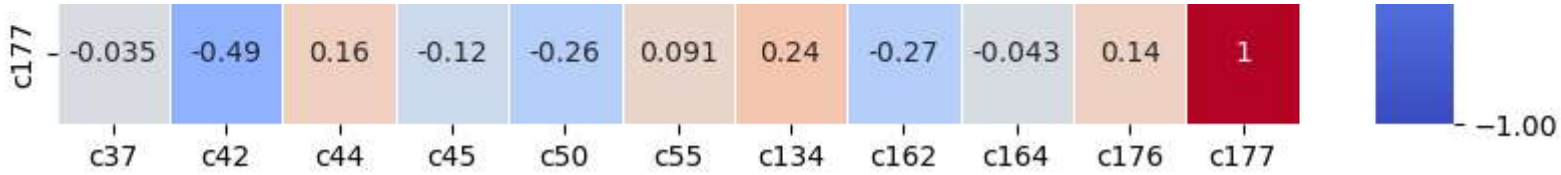
In [34]:
```python
feature_list=list(selected_features)
```

In [35]:
```python
# code to remove closely correlated columns to remove multicollineaarity
columns_to_remove = []
for i in range(len(feature_list)):
    for j in range(i+1, len(feature_list)):
        col1 = feature_list[i]
        col2 = feature_list[j]
        corr_coefficient = df[col1].corr(df[col2])

        if abs(corr_coefficient) >= 0.8:
            columns_to_remove.append(col2)
feature_list = [col for col in feature_list if col not in columns_to_remove]

print(feature_list)
```

```
['c37', 'c42', 'c44', 'c45', 'c50', 'c55', 'c134', 'c162', 'c164', 'c176', 'c177']
```

In [37]:
```python
# verification of non existence of multicollinearity by heat map
import seaborn as sns

correlation_matrix = df[feature_list].corr()
plt.figure(figsize=(10, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5,vmin=-1,vmax=1)
plt.title('Correlation Heatmap')
plt.show()
```


Correlation Heatmap
c37
c42
c44
c45
c50
c55
c134
c162
c164
c176
1 -0.16 0.63 0.51 0.57 0.63 0.6 0.74 0.63 0.64 -0.035
-0.16 1 -0.48 -0.08 -0.081 -0.11 -0.23 -0.16 -0.16 -0.4 -0.49
0.63 -0.48 1 0.48 0.45 0.45 0.34 0.66 0.49 0.59 0.16
0.51 -0.08 0.48 1 0.66 0.5 0.22 0.56 0.49 0.34 -0.12
0.57 -0.081 0.45 0.66 1 0.76 0.31 0.55 0.65 0.66 -0.26
0.63 -0.11 0.45 0.5 0.76 1 0.68 0.35 0.76 0.57 0.091
0.6 -0.23 0.34 0.22 0.31 0.68 1 0.27 0.74 0.25 0.24
0.74 -0.16 0.66 0.56 0.55 0.35 0.27 1 0.54 0.46 -0.27
0.63 -0.16 0.49 0.49 0.65 0.76 0.74 0.54 1 0.42 -0.043
0.64 -0.4 0.59 0.34 0.66 0.57 0.25 0.46 0.42 1 0.14
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75



In [17]:

```python
# model fitting and validation by classification report and vif values on test data
X=df[feature_list]
X=sm.add_constant(X)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=40)


model = LogisticRegression(solver='lbfgs', multi_class='multinomial',max_iter=1000)
model.fit(X_train, y_train)


y_pred = model.predict(X_test)


accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)

print("Accuracy:", accuracy)
print("Classification Report:")
print(classification_rep)

coefficients_df = pd.DataFrame({'Feature': X.columns, 'Coefficient': model.coef_[0]})
intercept = model.intercept_

print(coefficients_df)
print("Intercept:", intercept)
vif_data = pd.DataFrame()
vif_data["Variable"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

print(vif_data)
```

```
Accuracy: 0.9025974025974026
Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.93      0.91       168
           1       0.91      0.87      0.89       140

    accuracy                           0.90       308
   macro avg       0.90      0.90      0.90       308
weighted avg       0.90      0.90      0.90       308

   Feature  Coefficient
0    const    -0.000533
1      c37    -0.549707
2      c42    -2.617879
3      c44     0.790321
4      c45     1.551884
5      c50     0.037321
6      c55     1.723450
7     c134     1.214731
8     c162     1.005144
9     c164    -2.602028
10    c176    -0.112450
11    c177     0.393976
Intercept: [-3.18545612]
   Variable        VIF
0     const  76.509421
1       c37   6.846576
2       c42   2.295362
3       c44   3.039169
4       c45   2.546366
5       c50   7.594723
6       c55   6.821299
7      c134   5.929591
8      c162   4.812648
9      c164   5.015251
10     c176   4.821436
11     c177   2.482479
```

In [18]:
```
# modifing df to undersample 0 in alarm category
df = df.iloc[700:850]
df.head()
```

Out[18]:

	c2	c3	c4	c5	c6	c7	c8	c9	c10	c11	...	c237	c238	c239	c241	
700	0	0.156967	0.480511	0.435462	0.468608	0.209849	0.582494	0.542518	0.416656	0.227490	...	0	0.161308	0.486274	2.384177	12.
701	0	0.154743	0.469058	0.434439	0.478901	0.223319	0.576424	0.552099	0.426188	0.214063	...	0	0.176081	0.489614	2.376469	12.
702	0	0.152069	0.460364	0.431758	0.480070	0.234532	0.571259	0.562485	0.434812	0.198187	...	0	0.190804	0.485254	2.370145	12.
703	0	0.148534	0.447836	0.427451	0.484459	0.243162	0.567423	0.570813	0.443226	0.181495	...	0	0.190356	0.485273	2.364193	12.
704	0	0.144651	0.431641	0.422451	0.487771	0.250164	0.562221	0.577528	0.452046	0.163776	...	0	0.188855	0.493953	2.357963	12.

5 rows × 211 columns

In [27]:
```
# code to remove closely correlated columns to remove multicollineaarity
columns_to_remove = []
for i in range(len(feature_list)):
    for j in range(i+1, len(feature_list)):
        col1 = feature_list[i]
        col2 = feature_list[j]
        corr_coefficient = df[col1].corr(df[col2])

        if abs(corr_coefficient) >= 0.8:
            columns_to_remove.append(col2)
feature_list = [col for col in feature_list if col not in columns_to_remove]

print(feature_list)
```

```
['c37', 'c42', 'c44', 'c134', 'c164', 'c177']
```

In [29]:
```
# model fitting and validation by classification report and vif values on test data
X=df[feature_list]
X=sm.add_constant(X)
y = df['alarm']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)


model = LogisticRegression(solver='lbfgs', multi_class='multinomial',max_iter=1000)
model.fit(X_train, y_train)


y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)

print("Accuracy:", accuracy)
print("Classification Report:")
print(classification_rep)

coefficients_df = pd.DataFrame({'Feature': X.columns, 'Coefficient': model.coef_[0]})
intercept = model.intercept_

print(coefficients_df)
print("Intercept:", intercept)
vif_data = pd.DataFrame()
vif_data["Variable"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

print(vif_data)
```

```
Accuracy: 0.9333333333333333
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.95      0.93        22
           1       0.95      0.91      0.93        23

    accuracy                           0.93        45
   macro avg       0.93      0.93      0.93        45
weighted avg       0.93      0.93      0.93        45

  Feature  Coefficient
0   const    -0.000193
1     c37    -0.090747
2     c42     0.103127
3     c44    -0.954978
4    c134     1.179167
5    c164     2.194071
6    c177    -0.533896
Intercept: [-1.4522946]
  Variable          VIF
0    const  2133.278381
1      c37     8.907618
2      c42     5.061097
3      c44     9.551640
4     c134     7.348615
5     c164     7.202475
6     c177     8.983924
```

In [ ]:

MODEL-2

In [ ]:
```
import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

In [ ]:
```
df = pd.read_csv("D:\IITB\ppt\DS 203\categorised_e11.csv")
```

In [ ]:
```
X = df[['c26','c27','c28','c29','c30','c31','c32','c33','c39','c139','c142','c143','c155','c156','c157','c158','c160','c161','c16
```

In [ ]:
```
df['alert'] = 0
df['alarm']= 0
```

In [ ]:
```
#setting level as 0 in case of level being SAFE
#1 in case of level being MODERATE
#2 in case of level being HIGH
#3 in case of level being CRITICAL
```

In [ ]:
```
df['alarm'] = np.where((df['c51'] <= 5) & (df['c53'] <= 5), 0,
                np.where((df['c51'] <= 10) & (df['c53'] <= 10), 0,
                np.where((df['c51'] <= 20) & (df['c53'] <= 20), 0,1)))
df['alert'] = np.where((df['c51'] <= 5) & (df['c53'] <= 5), 0,
                np.where((df['c51'] <= 10) & (df['c53'] <= 10), 0,
                np.where((df['c51'] <= 20) & (df['c53'] <= 20), 1,0)))
```

In [ ]:
```
c = X.columns
col=list(c)
```

In [ ]:
```
#reducing multicollinearity by removing closely related columns
col_remove = []
for i in range(len(col)):
    for j in range(i+1,len(col)):
        col1 = col[i]
        col2 = col[j]
        corr_coeff = df[col1].corr(df[col2])
        if abs(corr_coeff) >= 0.65:
```

```
            col_remove.append(col2)

print(col_remove)
```

```
['c29', 'c142', 'c143', 'c155', 'c33', 'c143', 'c161', 'c163', 'c160', 'c161', 'c162', 'c161', 'c162', 'c163', 'c162', 'c163']
```

In [ ]:
```
col = [item for item in col if item not in col_remove]
print(col)
```

```
['c26', 'c27', 'c28', 'c30', 'c31', 'c32', 'c39', 'c139', 'c156', 'c157', 'c158']
```

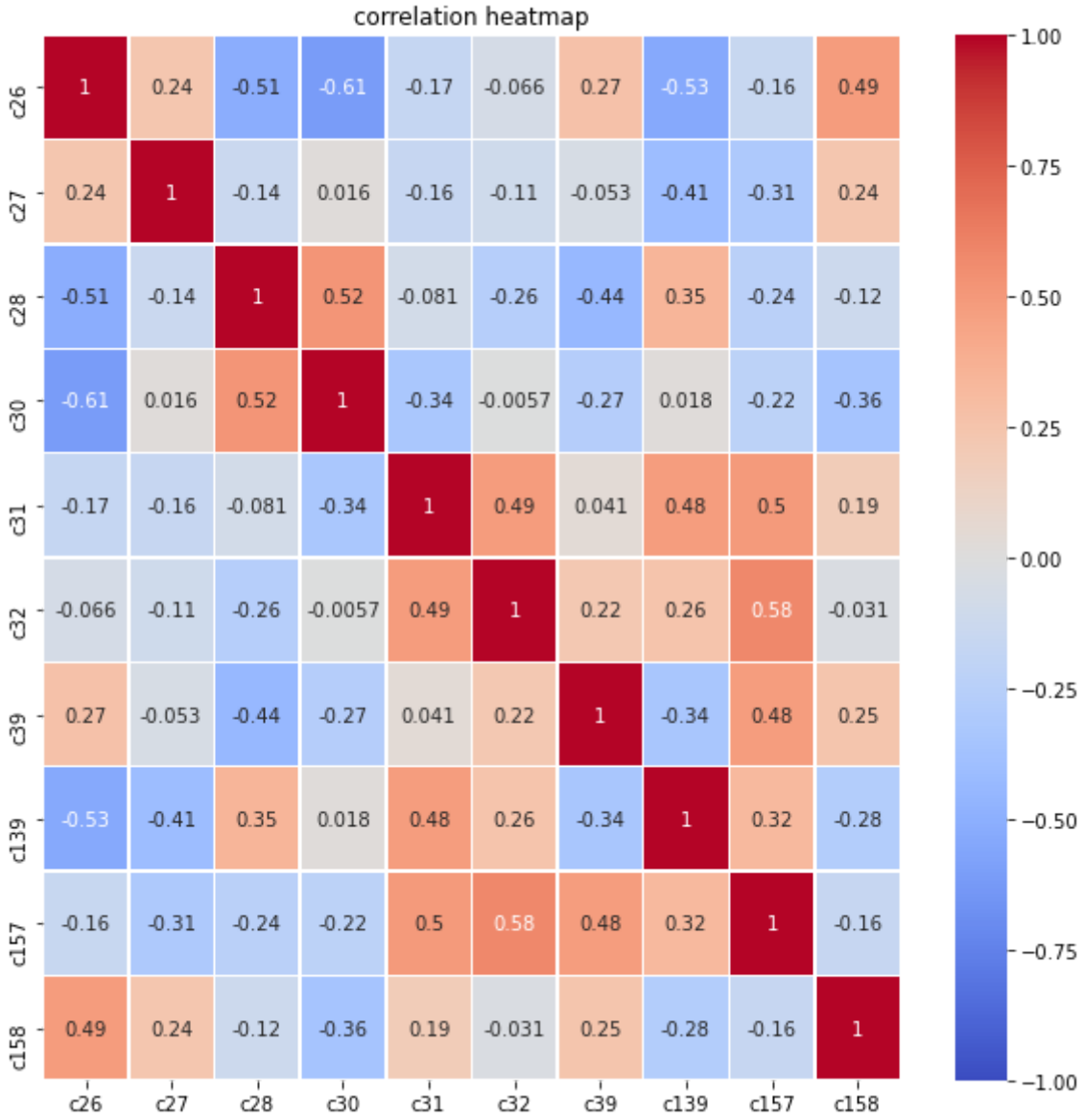
In [ ]:
```
#dropping column c156 as all values are 0
col.remove('c156')
print(col)
```

```
['c26', 'c27', 'c28', 'c30', 'c31', 'c32', 'c39', 'c139', 'c157', 'c158']
```

In [ ]:
```
#checking the multicollinearity of remaing columns
import matplotlib.pyplot as plt
import seaborn as sns
```

In [ ]:
```
corr_matrix = df[col].corr()
plt.figure(figsize = (10,10))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5, vmin=-1, vmax=+1, cbar=True)
plt.title('correlation heatmap')
plt.show
```

Out[ ]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



In [ ]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

In [ ]:
```
X = df[col]
X= sm.add_constant(X)
```

```
y = df['alert']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=59)
model = LogisticRegression(solver='lbfgs', multi_class='multinomial', max_iter=1000)
model.fit(X_train,y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test,y_pred)
class_repo = classification_report(y_test,y_pred)

print("Accuracy: ", accuracy,'\n')
print('Classification Report:','\n')
print(class_repo,'\n')

coeff_df = pd.DataFrame({'Feature':X.columns, 'Coefficients':model.coef_[0]})
intercept = model.intercept_

print(coeff_df)
print("Intercept: ", intercept,'\n')

vif_data = pd.DataFrame()
vif_data['Feature'] = X.columns
vif_data['VIF'] =  [variance_inflation_factor(X.values,i) for i in range(X.shape[1])]

print(vif_data)
```

```
Accuracy:  0.827922077922078

Classification Report:

              precision    recall  f1-score   support

           0       0.84      0.85      0.84       168
           1       0.82      0.80      0.81       140

    accuracy                           0.83       308
   macro avg       0.83      0.83      0.83       308
weighted avg       0.83      0.83      0.83       308


   Feature  Coefficients
0    const      0.000522
1      c26     -1.464785
2      c27     -1.428245
3      c28      1.496380
4      c30     -0.239924
5      c31      2.091384
6      c32     -1.547246
7      c39      3.369782
8     c139      0.685629
9     c157      0.079947
10    c158     -1.383131
Intercept:  [-1.72436812]

   Feature         VIF
0    const  238.705670
1      c26    4.175859
2      c27    1.345451
3      c28    2.331209
4      c30    4.059637
5      c31    2.883729
6      c32    2.405327
7      c39    2.500257
8     c139    3.086899
9     c157    3.281704
10    c158    2.067945
```

In [ ]:
```
coeff_df['Abs coeff'] = coeff_df['Coefficients'].abs()
sorted = coeff_df.sort_values(by = 'Abs coeff', ascending = False)
```

In [ ]:
```
print(sorted[['Feature', 'Abs coeff']])
```

```
   Feature  Abs coeff
7      c39   3.212716
5      c31   1.948652
2      c27   1.503808
3      c28   1.497426
6      c32   1.458192
1      c26   1.379747
10    c158   1.213619
8     c139   0.832173
4      c30   0.308649
9     c157   0.094550
0    const   0.000859
```

In [ ]:
```
# modifing df to undersample 0 in alarm category
df = df.iloc[700:850]
df.head()
```

Out[ ]:

	c2	c3	c4	c5	c6	c7	c8	c9	c10	c11	...	c237	c238	c239	c241	c51	
700	0	0.156967	0.480511	0.435462	0.468608	0.209849	0.582494	0.542518	0.416656	0.227490	...	0	0.161308	0.486274	2.384177	12.732287	9.6
701	0	0.154743	0.469058	0.434439	0.478901	0.223319	0.576424	0.552099	0.426188	0.214063	...	0	0.176081	0.489614	2.376469	12.757163	9.7
702	0	0.152069	0.460364	0.431758	0.480070	0.234532	0.571259	0.562485	0.434812	0.198187	...	0	0.190804	0.485254	2.370145	12.783161	9.7
703	0	0.148534	0.447836	0.427451	0.484459	0.243162	0.567423	0.570813	0.443226	0.181495	...	0	0.190356	0.485273	2.364193	12.805033	9.8
704	0	0.144651	0.431641	0.422451	0.487771	0.250164	0.562221	0.577528	0.452046	0.163776	...	0	0.188855	0.493953	2.357963	12.824723	9.9

5 rows × 211 columns

In [ ]:
```
X = df[['c26','c27','c28','c29','c30','c31','c32','c33','c39','c139','c142','c143','c155','c156','c157','c158','c160','c161','c16
c = X.columns
col=list(c)
```

In [ ]:
```
col_remove = []
for i in range(len(col)):
    for j in range(i+1,len(col)):
        col1 = col[i]
        col2 = col[j]
```

```
        corr_coeff = df[col1].corr(df[col2])
        if abs(corr_coeff) >= 0.65:
            col_remove.append(col2)

print(col_remove)
```

```
['c30', 'c32', 'c33', 'c39', 'c139', 'c142', 'c143', 'c142', 'c143', 'c158', 'c31', 'c161', 'c31', 'c32', 'c33', 'c39', 'c139',
'c142', 'c33', 'c39', 'c139', 'c142', 'c39', 'c139', 'c142', 'c139', 'c142', 'c142', 'c143', 'c158', 'c161']
```

In [ ]:
```
col = [item for item in col if item not in col_remove]
print(col)
```

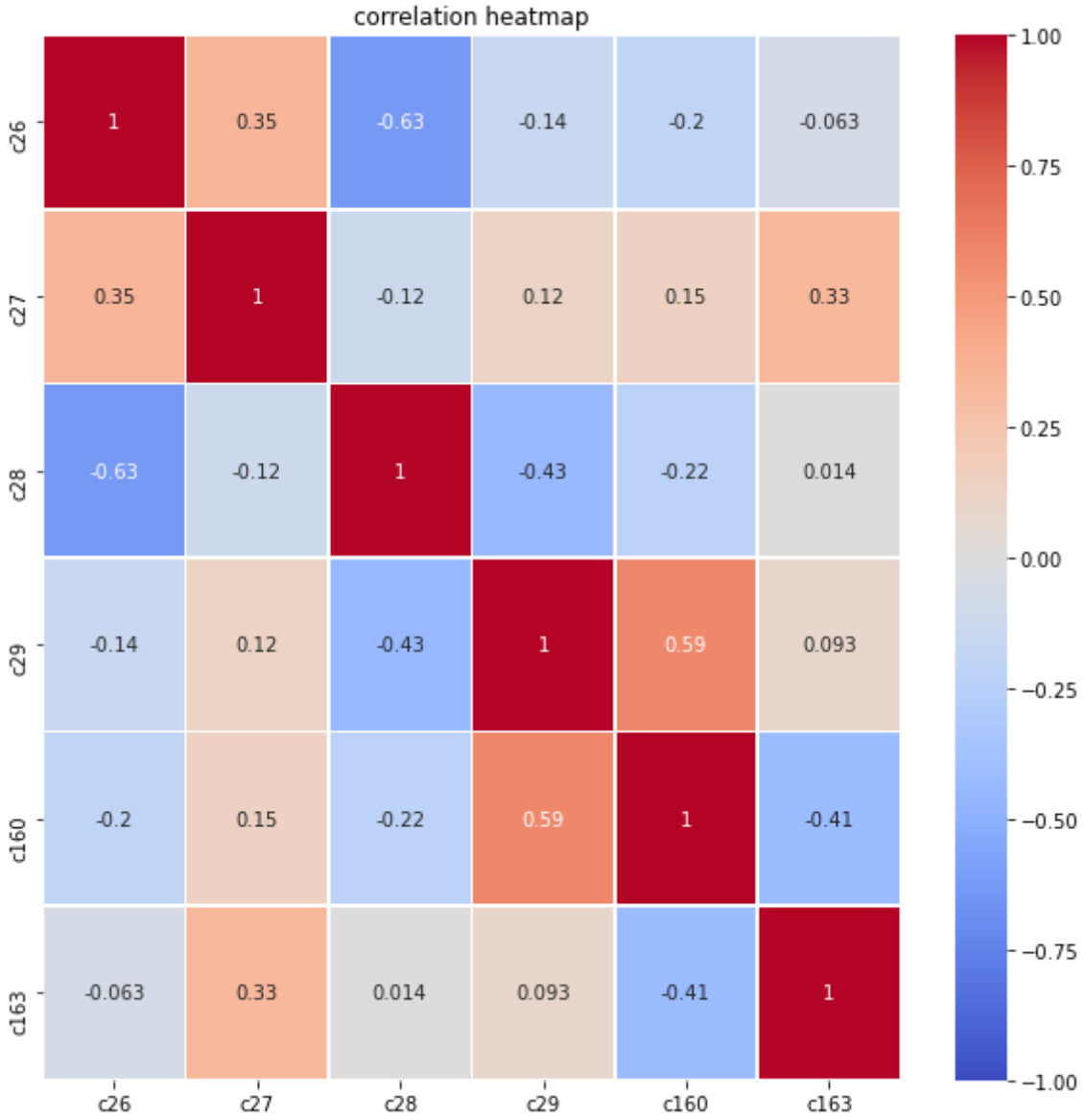
```
['c26', 'c27', 'c28', 'c29', 'c155', 'c156', 'c157', 'c160', 'c162', 'c163']
```

In [ ]:
```
# in c156 all value are 0 in c157 all are 1 in modified df
col.remove('c155')
col.remove('c156')
col.remove('c157')
col.remove('c162')
print(col)
```

```
['c26', 'c27', 'c28', 'c29', 'c160', 'c163']
```

In [ ]:
```
corr_matrix = df[col].corr()
plt.figure(figsize = (10,10))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5, vmin=-1, vmax=+1, cbar=True)
plt.title('correlation heatmap')
plt.show
```

Out[ ]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



In [ ]:

```
X = df[col]
X1= sm.add_constant(X)
y = df['alarm']

X_train, X_test, y_train, y_test = train_test_split(X1, y, test_size=0.2, random_state=42)
model = LogisticRegression(solver='lbfgs', multi_class='multinomial')
```

```
model.fit(X_train,y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test,y_pred)
class_repo = classification_report(y_test,y_pred)

print("Accuracy: ", accuracy,'\n')
print('Classification Report:','\n')
print(class_repo,'\n')

coeff_df = pd.DataFrame({'Feature':X1.columns, 'Coefficients':model.coef_[0]})
intercept = model.intercept_

print(coeff_df)
print("Intercept: ", intercept,'\n')

vif_data = pd.DataFrame()
vif_data['Feature'] = X1.columns
vif_data['VIF'] =  [variance_inflation_factor(X1.values,i) for i in range(X1.shape[1])]

print(vif_data)
```

```
Accuracy:  0.9

Classification Report:

              precision    recall  f1-score   support

           0       0.93      0.87      0.90        15
           1       0.88      0.93      0.90        15

    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30


  Feature  Coefficients
0   const     -0.000059
1     c26     -0.182188
2     c27      0.605887
3     c28      0.139320
4     c29     -0.157282
5    c160     -0.050321
6    c163      1.613156
Intercept:  [-1.5239538]

  Feature          VIF
0   const  2142.257240
1     c26     4.940162
2     c27     2.484846
3     c28     4.137935
4     c29     2.597159
5    c160     3.819811
6    c163     2.736565
```

In [ ]:
```
coeff_df['Abs coeff'] = coeff_df['Coefficients'].abs()
sorted = coeff_df.sort_values(by = 'Abs coeff', ascending = False)
```

In [ ]:
```
print(sorted[['Feature', 'Abs coeff']])
```

	Feature	Abs coeff
6	c163	1.613156
2	c27	0.605887
1	c26	0.182188
4	c29	0.157282
3	c28	0.139320
5	c160	0.050321
0	const	0.000059

MODEL 3(a)

In [171]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

In [172]:
```
df = pd.read_csv('categorised_e11.csv')
```

In [188]:
```
from sklearn.model_selection import cross_val_score

k_values = [30,35, 40,41,42,43,44,45,46, 47, 48, 49, 50]

for k in k_values:
    selector = SelectKBest(f_regression, k=k)
    x_train_selected = selector.fit_transform(x_train, y_train)

    scores = cross_val_score(LinearRegression(), x_train_selected, y_train, cv=5)
    print(f'k={k}, Mean Cross-Validation Score: {np.mean(scores)}')
```

```
k=30, Mean Cross-Validation Score: 0.9935992519710691
k=35, Mean Cross-Validation Score: 0.9962952355098869
k=40, Mean Cross-Validation Score: 0.9970920982811637
k=41, Mean Cross-Validation Score: 0.9970942352715533
k=42, Mean Cross-Validation Score: 0.9970800745191793
k=43, Mean Cross-Validation Score: 0.9973065832487261
k=44, Mean Cross-Validation Score: 0.9974241320173711
k=45, Mean Cross-Validation Score: 0.9975958596031494
k=46, Mean Cross-Validation Score: 0.9974903283055703
k=47, Mean Cross-Validation Score: 0.9971209440664477
k=48, Mean Cross-Validation Score: 0.9971960062912537
k=49, Mean Cross-Validation Score: 0.9973083042822593
k=50, Mean Cross-Validation Score: 0.9963712218155903
```

In [187]:
```
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

X = df.drop(columns=['c241'])
y = df['c241']

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=58)

k = 45
selector = SelectKBest(f_regression, k=k)
x_train_selected = selector.fit_transform(x_train, y_train)

selected_features = X.columns[selector.get_support()]

print('Selected Features:', selected_features)
```

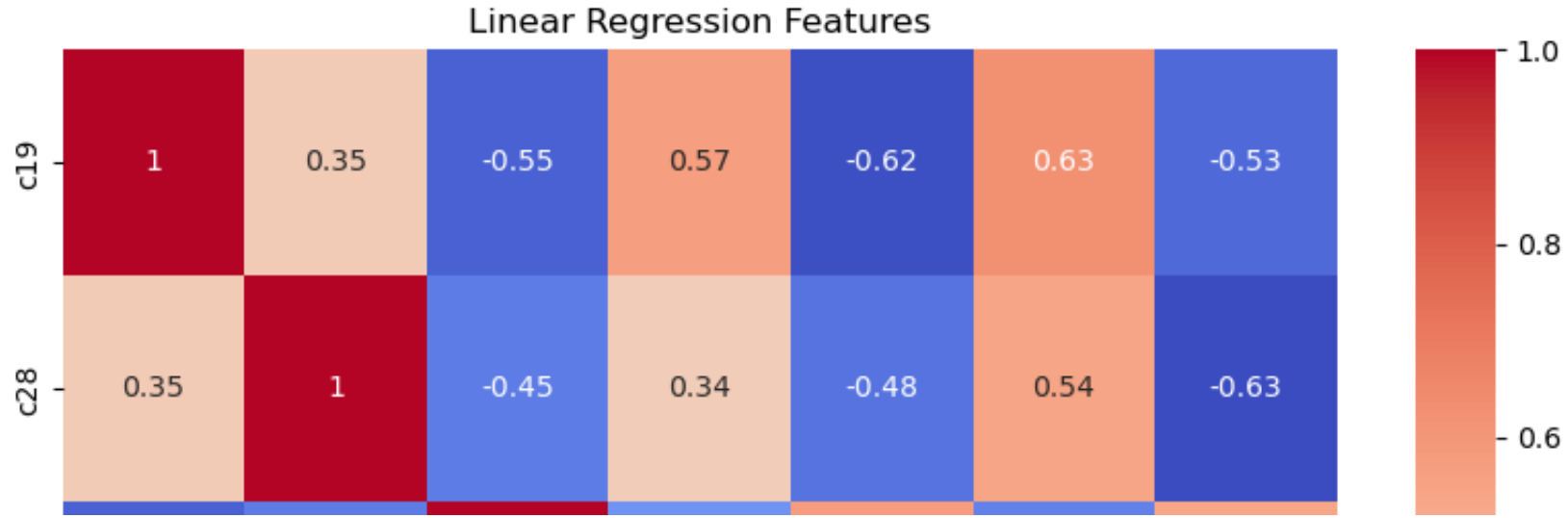
```
Selected Features: Index(['c19', 'c28', 'c29', 'c58', 'c60', 'c64', 'c67', 'c70', 'c74', 'c75',
       'c76', 'c77', 'c81', 'c83', 'c84', 'c88', 'c90', 'c94', 'c95', 'c101',
       'c103', 'c111', 'c115', 'c120', 'c133', 'c134', 'c139', 'c141', 'c142',
       'c143', 'c144', 'c125', 'c129', 'c151', 'c167', 'c173', 'c180', 'c181',
       'c186', 'c201', 'c203', 'c205', 'c224', 'c225', 'c227'],
      dtype='object')
```

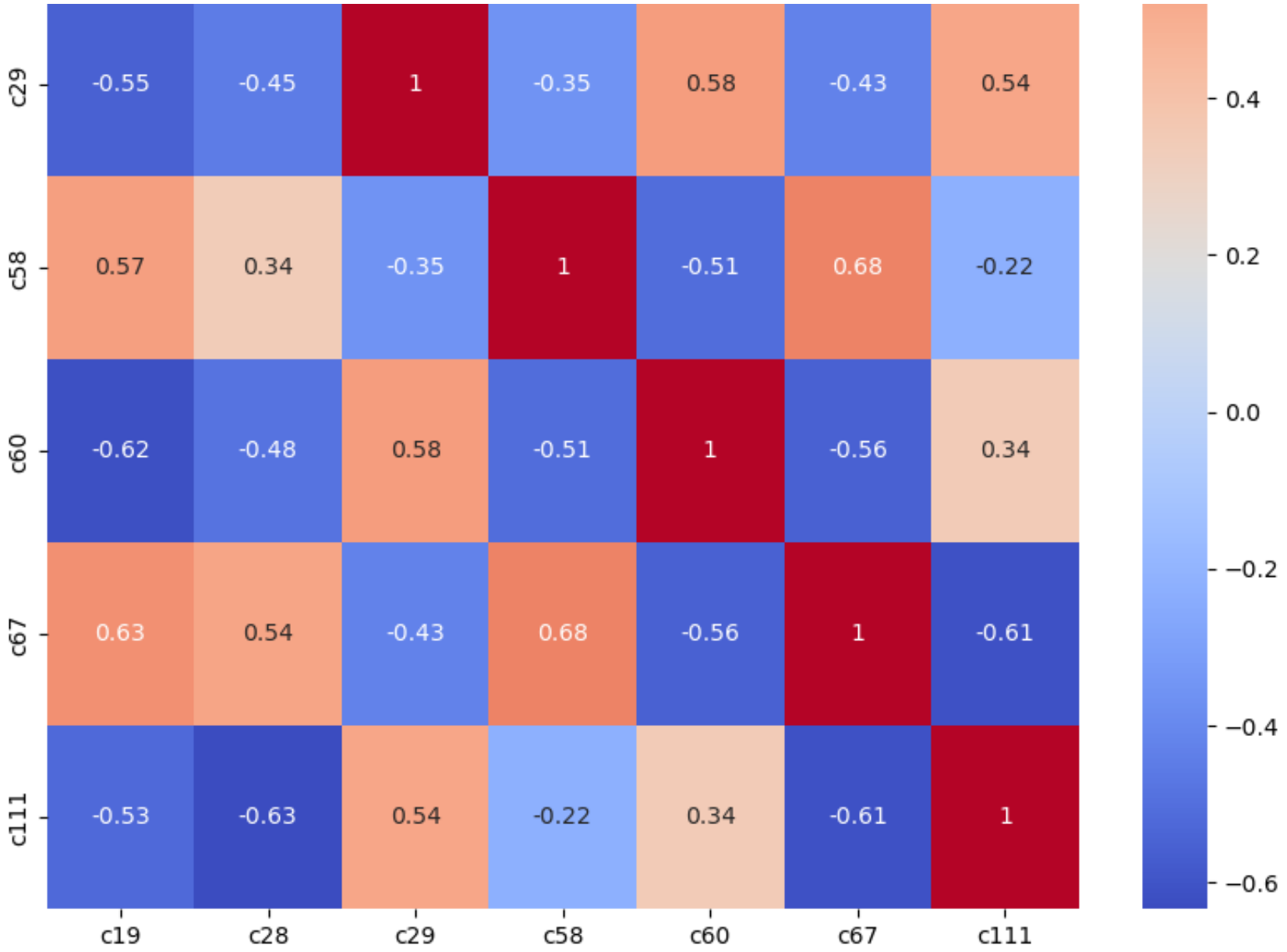
In [189]:
```
feature_list = list(selected_features)
```

In [190]:
```
columns_to_remove = []
for i in range(len(feature_list)):
    for j in range(i+1, len(feature_list)):
        col1 = feature_list[i]
        col2 = feature_list[j]
        corr_coefficient = df[col1].corr(df[col2])
        if abs(corr_coefficient) >= 0.71:
            columns_to_remove.append(col2)
feature_list = [col for col in feature_list if col not in columns_to_remove]
print(feature_list)
```

```
['c19', 'c28', 'c29', 'c58', 'c60', 'c67', 'c111']
```

In [191]:
```
import seaborn as sns
correlation_matrix = df[feature_list].corr()
plt.figure(figsize=(10,10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Linear Regression Features')
plt.show()
```




c29
c58
c60
c67
c111
-0.55 -0.45 1 -0.35 0.58 -0.43 0.54
0.57 0.34 -0.35 1 -0.51 0.68 -0.22
-0.62 -0.48 0.58 -0.51 1 -0.56 0.34
0.63 0.54 -0.43 0.68 -0.56 1 -0.61
-0.53 -0.63 0.54 -0.22 0.34 -0.61 1
c19 c28 c29 c58 c60 c67 c111
0.4
0.2
0.0
−0.2
−0.4
−0.6

In [102]:

In [192]:

```
from sklearn.metrics import mean_squared_error, r2_score

X = df[feature_list]
X =  sm.add_constant(X)
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
mlr_model = sm.OLS(y_train, x_train).fit()
print(mlr_model.summary())

y_pred = mlr_model.predict(x_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')
print(f'R-squared: {r2}')
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                   c241   R-squared:                       0.897
Model:                            OLS   Adj. R-squared:                  0.896
Method:                 Least Squares   F-statistic:                     880.7
Date:                Sat, 11 Nov 2023   Prob (F-statistic):               0.00
Time:                        15:46:35   Log-Likelihood:                 1531.8
No. Observations:                 717   AIC:                            -3048.
Df Residuals:                     709   BIC:                            -3011.
Df Model:                           7
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          2.0053      0.010    194.815      0.000       1.985       2.025
c19            0.1216      0.007     18.554      0.000       0.109       0.134
c28            0.2242      0.008     28.240      0.000       0.209       0.240
c29           -0.0296      0.007     -4.115      0.000      -0.044      -0.015
c58            0.1387      0.007     20.274      0.000       0.125       0.152
c60           -0.0265      0.006     -4.127      0.000      -0.039      -0.014
```

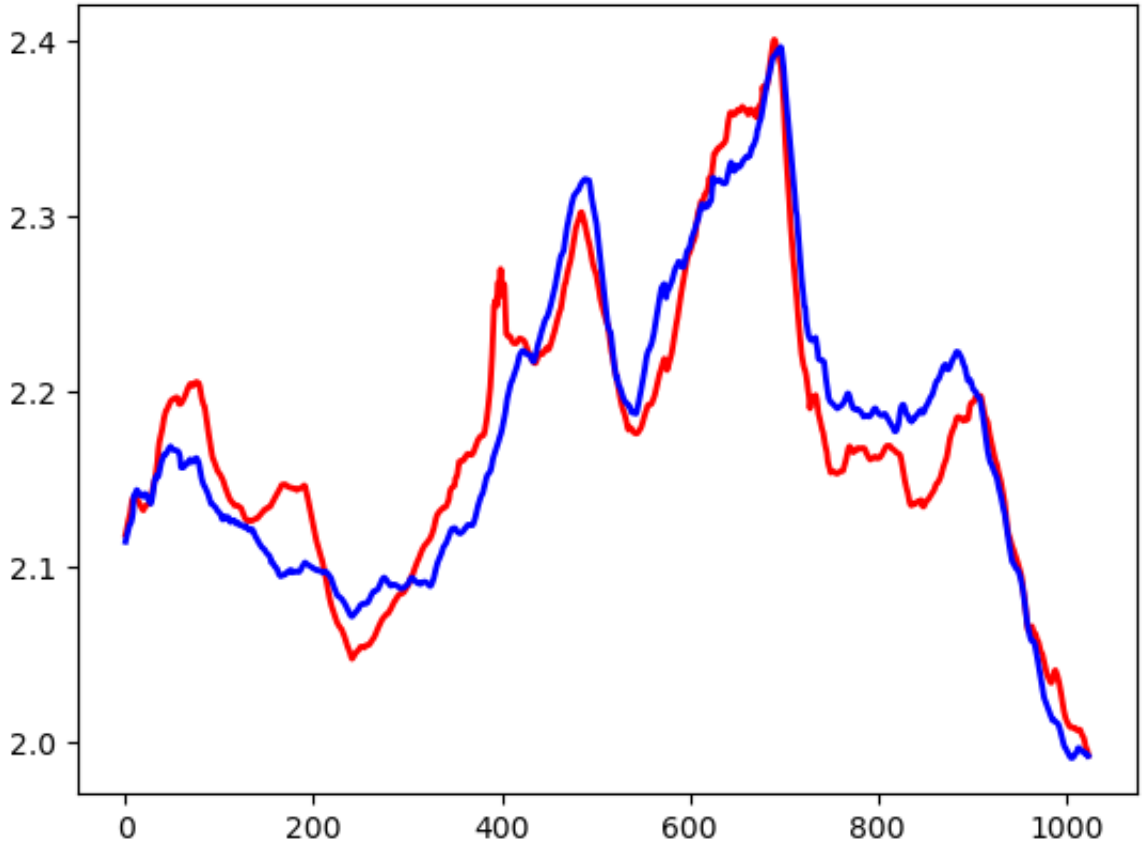
```
c67          -0.0956      0.011     -8.791      0.000      -0.117      -0.074
c111         -0.0251      0.008     -2.976      0.003      -0.042      -0.009
==============================================================================
Omnibus:                        6.909   Durbin-Watson:                   1.905
Prob(Omnibus):                  0.032   Jarque-Bera (JB):                6.842
Skew:                          -0.211   Prob(JB):                       0.0327
Kurtosis:                       3.227   Cond. No.                         22.6
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Mean Squared Error: 0.0007574664154717774
R-squared: 0.904392171404711
```

In [193]:
```
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Assuming you already have x_train as your design matrix with all the predictor variables

# Calculate VIF for each column in x_train
vif_data = pd.DataFrame()
vif_data["Variable"] = x_train.columns
vif_data["VIF"] = [variance_inflation_factor(x_train.values, i) for i in range(x_train.shape[1])]

# Display the VIF data
print(vif_data)
```

```
  Variable        VIF
0    const  92.024929
1      c19   2.605697
2      c28   2.122940
3      c29   2.031440
4      c58   2.363970
5      c60   2.402207
6      c67   3.468163
7     c111   3.206809
```

In [195]:
```
y_total_pred = mlr_model.predict(X)
plt.plot(y_total_pred, color='red', linestyle='-', linewidth=2)
plt.plot(y, color='blue', linestyle='-', linewidth=2)
```

Out[195]: [<matplotlib.lines.Line2D at 0x153f34c10>]



In [ ]:

MODEL 3(b)

In [111]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

In [112]:
```
df = pd.read_csv('categorised_e11.csv')
```

In [113]:
```
a= list(df.columns)
print(a, np.size(a))
```

```
['Unnamed: 0', 'c2', 'c3', 'c4', 'c5', 'c6', 'c7', 'c8', 'c9', 'c10', 'c11', 'c12', 'c13', 'c14', 'c15'
, 'c16', 'c17', 'c18', 'c19', 'c20', 'c21', 'c22', 'c23', 'c24', 'c25', 'c26', 'c27', 'c28', 'c29', 'c3
0', 'c31', 'c32', 'c33', 'c34', 'c35', 'c36', 'c37', 'c38', 'c39', 'c40', 'c41', 'c42', 'c43', 'c44', '
c45', 'c46', 'c47', 'c48', 'c49', 'c50', 'c55', 'c56', 'c57', 'c58', 'c59', 'c60', 'c61', 'c62', 'c63',
'c64', 'c65', 'c66', 'c67', 'c68', 'c69', 'c70', 'c71', 'c72', 'c73', 'c74', 'c75', 'c76', 'c77', 'c78'
, 'c79', 'c80', 'c81', 'c82', 'c83', 'c84', 'c85', 'c86', 'c87', 'c88', 'c89', 'c90', 'c91', 'c92', 'c9
3', 'c94', 'c95', 'c96', 'c97', 'c98', 'c99', 'c100', 'c101', 'c102', 'c103', 'c104', 'c105', 'c106', '
c107', 'c108', 'c109', 'c110', 'c111', 'c112', 'c113', 'c114', 'c115', 'c116', 'c117', 'c118', 'c119',
'c120', 'c121', 'c122', 'c123', 'c124', 'c133', 'c134', 'c135', 'c136', 'c137', 'c138', 'c139', 'c140',
'c141', 'c142', 'c143', 'c144', 'c145', 'c146', 'c147', 'c148', 'c149', 'c125', 'c126', 'c127', 'c128',
'c129', 'c130', 'c131', 'c132', 'c150', 'c151', 'c152', 'c153', 'c154', 'c155', 'c156', 'c157', 'c158',
'c159', 'c160', 'c161', 'c162', 'c163', 'c164', 'c165', 'c166', 'c167', 'c168', 'c169', 'c170', 'c171',
'c172', 'c173', 'c174', 'c175', 'c176', 'c177', 'c178', 'c179', 'c180', 'c181', 'c182', 'c183', 'c184',
'c185', 'c186', 'c187', 'c191', 'c192', 'c193', 'c194', 'c195', 'c196', 'c197', 'c198', 'c200', 'c201',
'c203', 'c205', 'c224', 'c225', 'c227', 'c228', 'c230', 'c235', 'c236', 'c237', 'c238', 'c239', 'c241',
'c51', 'c52', 'c53', 'c54'] 210
```

In [114]:
```
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

X = df.drop(columns=['c241', 'c26', 'c27', 'c28', 'c29', 'c30', 'c31', 'c32',
'c33', 'c39', 'c139', 'c142', 'c143', 'c155', 'c156', 'c157', 'c158', 'c160', 'c161', 'c162', 'c163'])
y = df['c241']

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

#k = 45
#selector = SelectKBest(f_regression, k=k)
#x_train_selected = selector.fit_transform(x_train, y_train)

#selected_features = X.columns[selector.get_support()]

#print('Selected Features:', selected_features)
```

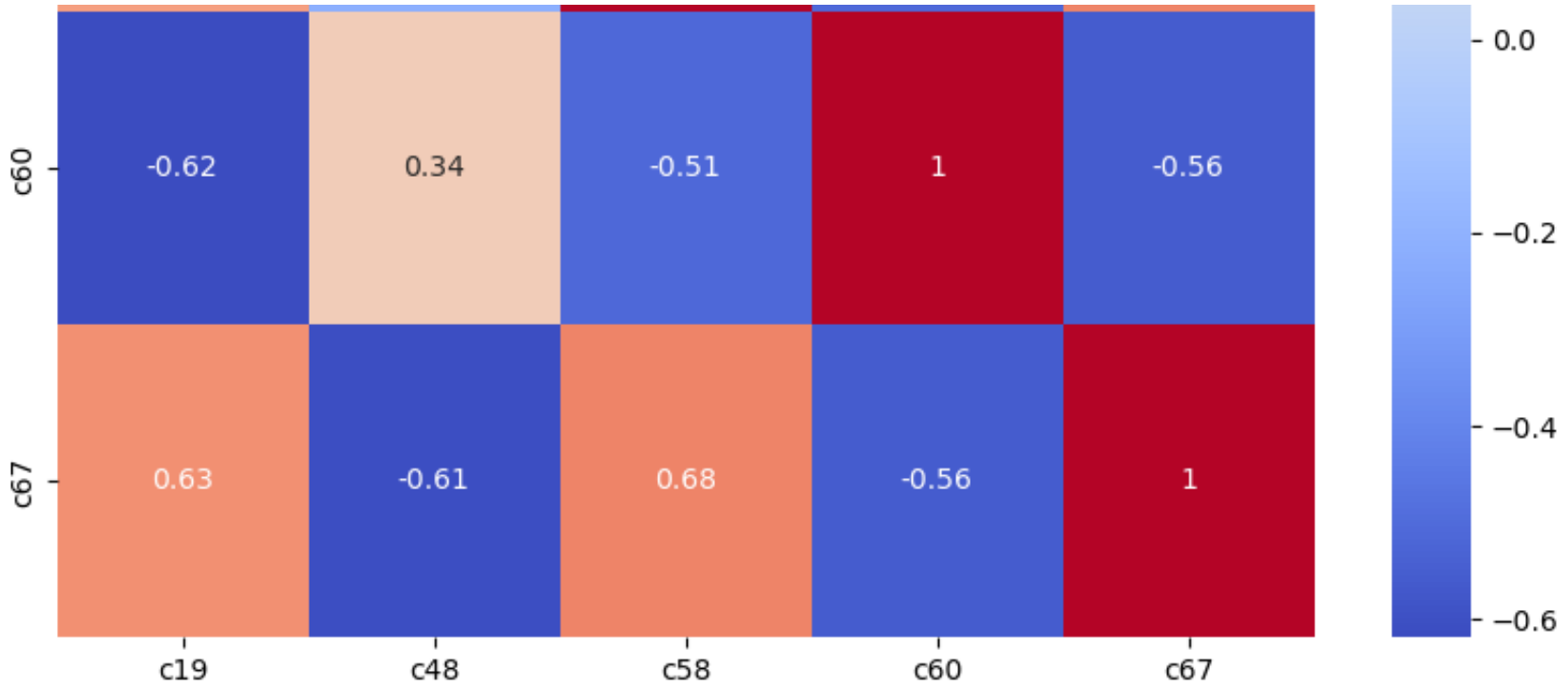
In [115]:
```
feature_list = list(selected_features)
```

In [116]:
```
columns_to_remove = []
for i in range(len(feature_list)):
    for j in range(i+1, len(feature_list)):
        col1 = feature_list[i]
        col2 = feature_list[j]
        corr_coefficient = df[col1].corr(df[col2])
        if abs(corr_coefficient) >= 0.71:
            columns_to_remove.append(col2)
feature_list = [col for col in feature_list if col not in columns_to_remove]
print(feature_list)
```

```
['c19', 'c48', 'c58', 'c60', 'c67']
```

In [117]:

```
import seaborn as sns
correlation_matrix = df[feature_list].corr()
plt.figure(figsize=(10,10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Linear Regression Features')
plt.show()
```


c60
c67
-0.62
0.34
-0.51
1
-0.56
0.63
-0.61
0.68
-0.56
1
c19
c48
c58
c60
c67
0.0
−0.2
−0.4
−0.6

In [118]:

```
from sklearn.metrics import mean_squared_error, r2_score

X = df[feature_list]
X =  sm.add_constant(X)
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)
mlr_model = sm.OLS(y_train, x_train).fit()
print(mlr_model.summary())

y_pred = mlr_model.predict(x_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')
print(f'R-squared: {r2}')
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                   c241   R-squared:                       0.765
Model:                            OLS   Adj. R-squared:                  0.764
Method:                 Least Squares   F-statistic:                     463.6
Date:                Sat, 11 Nov 2023   Prob (F-statistic):          5.89e-221
Time:                        20:36:27   Log-Likelihood:                 1254.8
No. Observations:                 717   AIC:                            -2498.
Df Residuals:                     711   BIC:                            -2470.
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          2.2020      0.011    204.976      0.000       2.181       2.223
c19            0.0712      0.009      7.709      0.000       0.053       0.089
c48           -0.1405      0.010    -13.951      0.000      -0.160      -0.121
c58            0.1662      0.010     16.492      0.000       0.146       0.186
c60           -0.0972      0.008    -12.077      0.000      -0.113      -0.081
c67           -0.0835      0.016     -5.201      0.000      -0.115      -0.052
```

```
==============================================================================
Omnibus:                        7.984   Durbin-Watson:                   2.161
Prob(Omnibus):                  0.018   Jarque-Bera (JB):                8.127
Skew:                          -0.260   Prob(JB):                       0.0172
Kurtosis:                       2.949   Cond. No.                         17.8
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Mean Squared Error: 0.0016600202362566444
R-squared: 0.8145018994426332
```

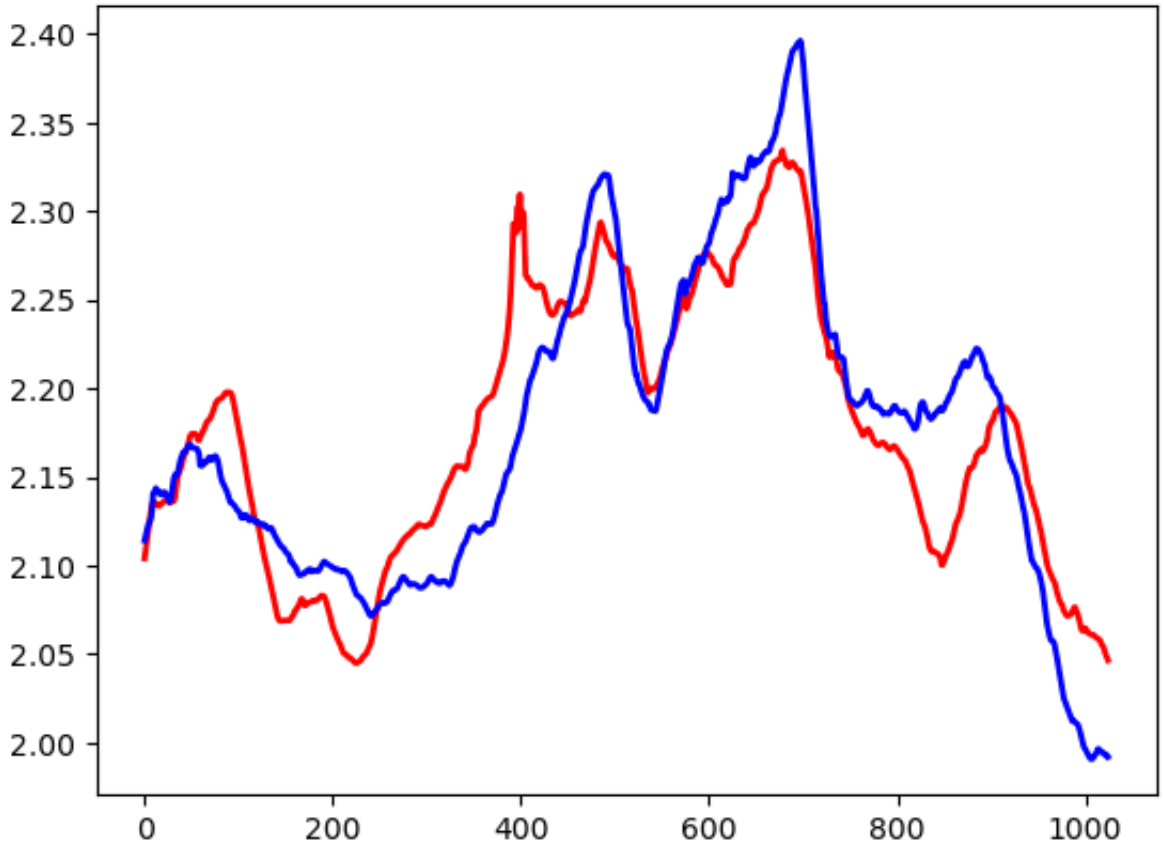
In [119]:
```
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Assuming you already have x_train as your design matrix with all the predictor variables

# Calculate VIF for each column in x_train
vif_data = pd.DataFrame()
vif_data["Variable"] = x_train.columns
vif_data["VIF"] = [variance_inflation_factor(x_train.values, i) for i in range(x_train.shape[1])]

# Display the VIF data
print(vif_data)
```

```
  Variable        VIF
0    const  46.419906
1      c19   2.312282
2      c48   2.086408
3      c58   2.360879
4      c60   1.770831
5      c67   3.438314
```

In [121]:
```
y_total_pred = mlr_model.predict(X)
```

In [123]:
```
plt.plot(y_total_pred, color='red', linestyle='-', linewidth=2)
plt.plot(y, color='blue', linestyle='-', linewidth=2)
```

Out[123]: [<matplotlib.lines.Line2D at 0x143069950>]



In [ ]: